

第三章 词法分析

许畅

南京大学计算机学院

2026年春季

内容

- 词法分析器的作用
- 词法单元的规约 (正则表达式)
- 词法单元的识别 (状态转换图)
- 词法分析器生成工具及设计
- 有穷自动机

词法分析器的作用

- 读入字符流，组成词素，输出**词法单元**序列
- 过滤空白、换行、制表符、注释等
- 将词素添加到符号表中
- 在逻辑上独立于语法分析，但是通常和语法分析器处于同一趟中

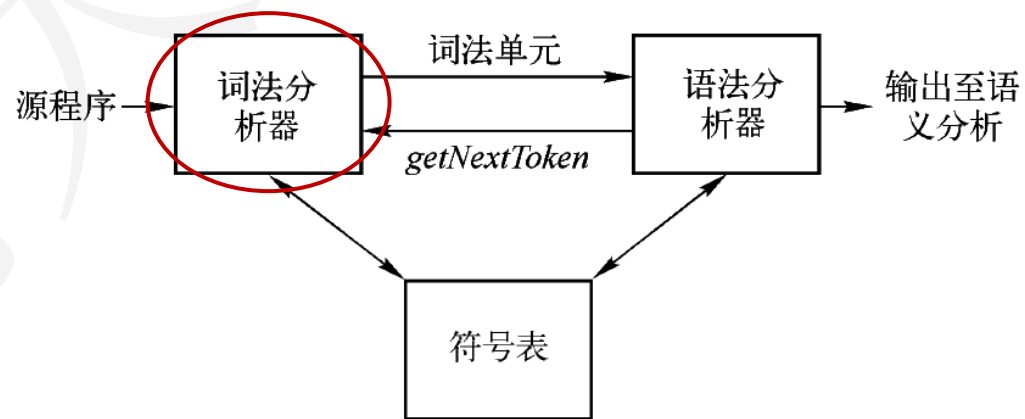


图 3-1 词法分析器与语法分析器之间的交互

为什么要设立独立的词法分析器？

- 简化编译器的设计
 - 词法分析器可以首先完成一些简单的处理工作
- 提高编译器效率
 - 相对于语法分析，词法分析过程简单，可高效实现(下推自动机 vs. 有穷自动机)
- 增强编译器的可移植性

词法单元、模式、词素

- 词法单元 (token)
 - <词法单元名、属性值 (可选)>
 - 单元名是表示词法单位种类的抽象符号，语法分析器通过单元名即可确定词法单元序列的结构
 - 属性值通常用于语义分析之后的阶段
- 模式 (pattern)
 - 描述了一类词法单元的词素可能具有的形式
- 词素 (lexeme)
 - 源程序中的字符序列
 - 它和某个词法单元的模式匹配，被词法分析器识别为该词法单元的实例

词法单元、模式、词素 (例子)

- `printf("Total = %d\n", score);`
 - `printf`和`score`与标识符 (id) 的模式匹配
 - `"Total = %d\n"`与literal的模式匹配

词法单元	非正式描述	词素示例
if	字符 i, f	if
else	字符 e, l, s, e	else
comparison	< 或 > 或 <= 或 >= 或 == 或 !=	<=, !=
id	字母开头的字母 / 数字串	Pi, score, D2
number	任何数字常量	3.14159, 0, 6.02e23
literal	在两个 "之间, 除" 以外的任何字符	"core dumped"

图 3-2 词法单元的例子

词法单元的属性

- 一个模式匹配多个词素时，必须通过属性来传递附加的信息
 - 属性值将被用于语义分析、代码生成等阶段
- 不同的目的需要不同的属性
 - 属性值通常是一个结构化数据
- 如词法单元id的属性
 - 词素、类型、第一次出现的位置 ...

内容

- 词法分析器的作用
- 词法单元的规约 (正则表达式)
- 词法单元的识别 (状态转换图)
- 词法分析器生成工具及设计
- 有穷自动机

词法单元的规约

- 正则表达式可以高效、简洁地描述处理词法单元时用到的模式类型
- 内容
 - 串和语言
 - 语言上的运算
 - 正则表达式和正则定义
 - 正则表达式的扩展

串和语言 (1)

- 字母表 (alphabet) : 一个有穷的符号集合
 - 符号典型例子: 字母、数字、标点符号
 - 例子: $\{0, 1\}$, ASCII, Unicode
 - 在理论上, 我们可以把任意的有限集合看作字母表
- 字母表上的串 (string) 是该表中符号的有穷序列
 - 串 s 的长度, 即 $|s|$, 是指 s 中符号出现的次数
 - 空串: 长度为 0 的串, ϵ
- 语言 (language) 是某个给定字母表上的串的可数集合

串和语言 (2)

- 和串有关的术语 (以banana为例)
 - 前缀 (prefix): 从串的尾部删除0个或多个符号后得到的串 (**ban**、banana、 ϵ)
 - 后缀 (suffix): 从串的开始处删除0个或多个符号后得到的串 (**nana**、banana、 ϵ)
 - 子串 (substring): 删除串的某个前缀和某个后缀得到的串 (banana、**nan**、 ϵ)
 - 真前缀、真后缀、真子串: 既不等于原串, 也不等于空串的前缀、后缀、子串 (前面例子的红色部分)
 - 子序列: 从原串中删除0个或多个符号后得到的串 (baan)

串和语言 (3)

- 串的运算
 - 连接 (concatenation): x 和 y 的连接是把 y 附加到 x 的后面而形成的串, 记作 xy
 - $x = \text{dog}, y = \text{house}, xy = \text{doghouse}$
 - 指数运算: $s^0 = \varepsilon, s^1 = s, s^i = s^{i-1}s$
 - $x = \text{dog}, x^0 = \varepsilon, x^1 = \text{dog}, x^3 = \text{dogdogdog}$

串和语言 (4)

- 语言的运算
 - 语言是某个给定字母表上的串的可数集合

运算	定义和表示
L 和 M 的并	$L \cup M = \{s \mid s \text{ 属于 } L \text{ 或者 } s \text{ 属于 } M\}$
L 和 M 的连接	$LM = \{st \mid s \text{ 属于 } L \text{ 且 } t \text{ 属于 } M\}$
L 的 Kleene 闭包	$L^* = \bigcup_{i=0}^{\infty} L^i$
L 的正闭包	$L^+ = \bigcup_{i=1}^{\infty} L^i$

图 3-6 语言上的运算的定义

串和语言 (5)

- 例子

- $L = \{ A, B, \dots, Z, a, b, \dots, z \}$
- $D = \{ 0, 1, \dots, 9 \}$
- $L \cup D = \{ A, B, \dots, Z, a, b, \dots, z, 0, 1, \dots, 9 \}$
- LD: 520个长度为2的串的集合 (字母 + 数字)
- L^4 : 所有由四个字母构成的串的集合
- L^* : 所有字母构成的集合, 包括 ϵ
- $L(L \cup D)^*$: ?
- D^+ : ?

正则表达式

- 字母表 Σ 上的正则表达式的定义
 - 基本部分
 - ε 是一个正则表达式, $L(\varepsilon) = \{ \varepsilon \}$
 - 如果 a 是 Σ 上的一个符号, 那么 a 是正则表达式, $L(a) = \{ a \}$
 - 归纳步骤
 - 选择: $(r) \mid (s)$, $L((r) \mid (s)) = L(r) \cup L(s)$
 - 连接: $(r)(s)$, $L((r)(s)) = L(r)L(s)$
 - 闭包: $(r)^*$, $L((r)^*) = (L(r))^*$
 - 括号: (r) , $L((r)) = L(r)$
- 运算的优先级: $*$ **>** 连接 **>** $\mid$, 如 $(a) \mid ((b)^*(c)) = a \mid b^*c$
- 正则集合 (regular set): 可用一个正则表达式定义的语言

例子

- $\Sigma = \{ a, b \}$
- $L(a \mid b) = \{ a, b \}$
- $L((a \mid b) (a \mid b)) = \{ aa, ab, ba, bb \}$
- $L(a^*) = \{ \varepsilon, a, aa, aaa, aaaa, \dots \}$
- $L((a \mid b)^*) = \{ \varepsilon, a, b, aa, ab, ba, bb, aaa, aab, \dots \}$
- $L(a \mid a^*b) = \{ a, b, ab, aab, aaab, \dots \}$

正则表达式的等价

- 如果 $L(r) = L(s)$ ，正则表达式 r 和 s 等价 (equivalent)

定律	描述
$r s = s r$	是可以交换的
$r (s t) = (r s) t$	是可结合的
$r(st) = (rs)t$	连接是可结合的
$r(s t) = rs rt; (s t)r = sr tr$	连接对 是可分配的
$\epsilon r = r\epsilon = r$	ϵ 是连接的单位元
$r^* = (r \epsilon)^*$	闭包中一定包含 ϵ
$r^{**} = r^*$	*具有幂等性

图 3-7 正则表达式的代数定律

正则定义 (1)

- 为了书写方便，可以给正则表达式命名
- 正则定义 (regular definition) 是如下的定义序列：
$$d_1 \rightarrow r_1$$
$$d_2 \rightarrow r_2$$
$$\dots$$
$$d_n \rightarrow r_n$$
- 其中
 - d_i 不在 Σ 中，且各不相同
 - 每个 r_i 是字母表 $\Sigma \cup \{d_1, d_2, \dots, d_{i-1}\}$ 上的正则表达式；这保证了不会出现递归定义

正则定义 (2)

- 各个 d_i 的 Σ 上的正则表达式如下
 - d_1 的正则表达式即 r_1
 - 将 r_2 中的 d_1 替换为 r_1 , 得到 d_2 的正则表达式
 - ...
 - 将 r_i 中的 $d_1, d_2, \dots, d_{i-1}$ 替换为各自的正则表达式, 得到 d_i 的正则表达式

例子

- C语言标识符的正则定义
 - $letter_ \rightarrow A \mid B \mid \dots \mid Z \mid a \mid b \mid \dots \mid z \mid _$
 - $digit \rightarrow 0 \mid 1 \mid \dots \mid 9$
 - $id \rightarrow letter_ (letter_ \mid digit)^*$
- id 对应的正则表达式为
 - $(A \mid B \mid \dots \mid Z \mid a \mid b \mid \dots \mid z \mid _)((A \mid B \mid \dots \mid Z \mid a \mid b \mid \dots \mid z \mid _) \mid (0 \mid 1 \mid \dots \mid 9))^*$

正则表达式的扩展

- 基本运算符：选择、连接、Kleene闭包
- 扩展的运算符
 - 一个或多个实例：单目后缀⁺
 - r^+ 等价于 rr^*
 - 零个或一个实例：?
 - $r?$ 等价于 $\epsilon \mid r$
 - 字符类
 - $[a_1a_2\dots a_n]$ 等价于 $a_1 \mid a_2 \mid \dots \mid a_n$
 - 使用-符号，如： $[a-e]$ 等价于 $a \mid b \mid c \mid d \mid e$
- 使正则表达式更简洁，但不会使其描述能力增强

内容

- 词法分析器的作用
- 词法单元的规约 (正则表达式)
- 词法单元的识别 (状态转换图)
- 词法分析器生成工具及设计
- 有穷自动机

词法单元的识别

- 词法分析器要求能够检查输入字符串，在其前缀中找出和某个模式匹配的词素
- 首先通过正则定义来描述各种词法单元的模式
- 定义 $ws \rightarrow (\text{blank} \mid \text{tab} \mid \text{newline})^+$ 来消除空白
 - 当词法分析器识别出这个模式时，**不返回**词法单元，继续识别其它模式

<i>digit</i>	→	[0-9]
<i>digits</i>	→	<i>digit</i> ⁺
<i>number</i>	→	<i>digits</i> (. <i>digits</i>)? (E [+-]? <i>digits</i>)?
<i>letter</i>	→	[A-Za-z]
<i>id</i>	→	<i>letter</i> (<i>letter</i> <i>digit</i>)*
<i>if</i>	→	if
<i>then</i>	→	then
<i>else</i>	→	else
<i>relop</i>	→	< > <= >= = <>

图 3-11 例 3.8 中词法单元的模式

词素	词法单元名字	属性值
Any <i>ws</i>	—	—
if	if	—
then	then	—
else	else	—
Any <i>id</i>	id	指向符号表条目的指针
Any <i>number</i>	number	指向符号表条目的指针
<	relop	LT
<=	relop	LE
=	relop	EQ
<>	relop	NE
>	relop	GT
>=	relop	GE

图 3-12 词法单元、它们的模式以及属性值

状态转换图

- 词法分析器的重要组成部分之一
- 状态转换图 (transition diagram)
 - 状态 (state): 表示在识别词素时可能出现的情况
 - 状态看作是已处理部分的总结
 - 某些状态为**接受状态**或**最终状态**, 表明已找到词素
 - 加上*的接受状态表示最后读入的符号不在词素中
 - **开始状态 (初始状态)**: 用Start边表示
 - 边 (edge): 从一个状态指向另一个状态
 - 边的标号是一个或多个符号
 - 当前状态为s, 下一个输入符号为a, 就沿着从s离开, 标号为a的边到达下一个状态

例子

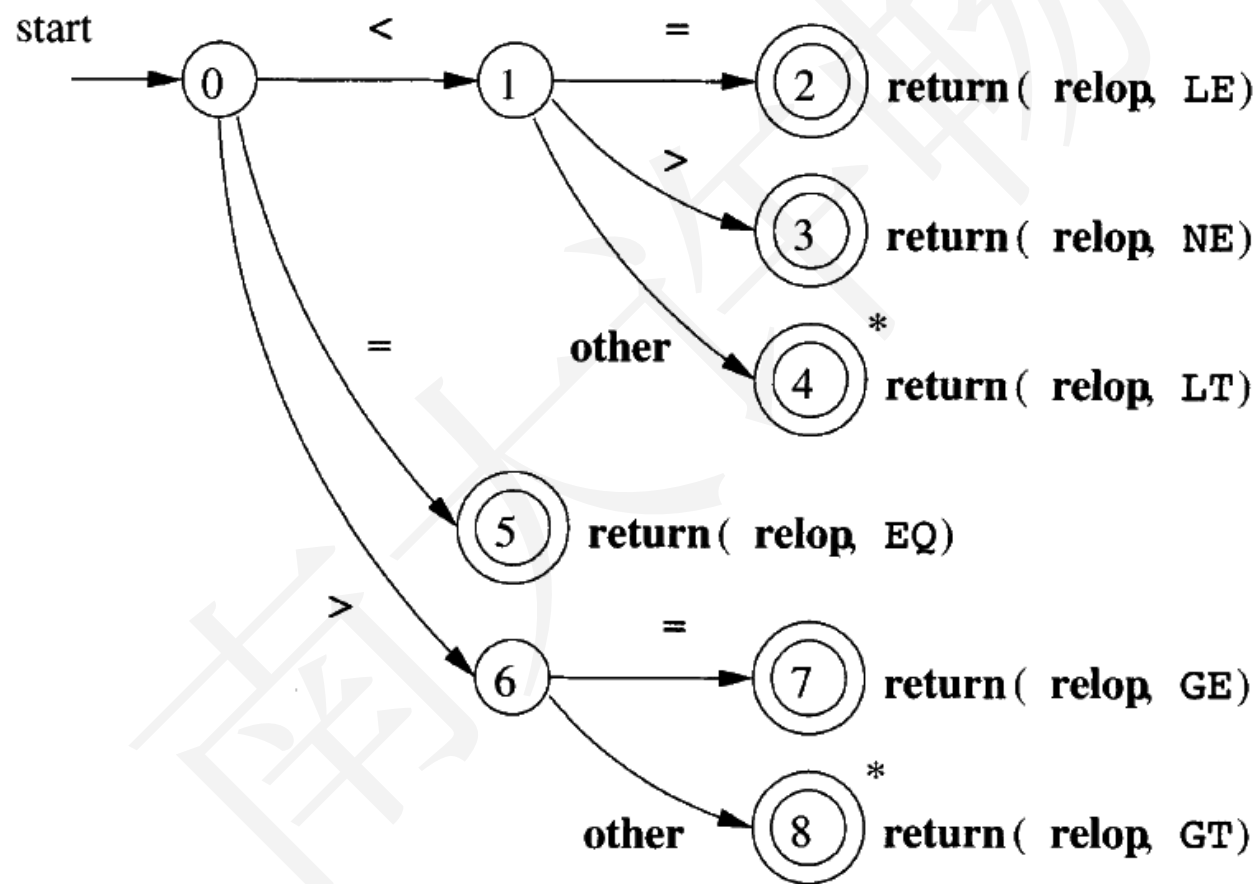


图 3-13 词法单元 **relop** 的状态转换图

保留字和标识符的识别

- 在很多时候，保留字也符合标识符的模式
 - 识别标识符的状态转换图也会识别保留字
- 解决方法
 - 在符号表中**先填保留字**，并指明它们不是普通标识符
 - 为保留字建立独立的、**高优先级**的状态转换图

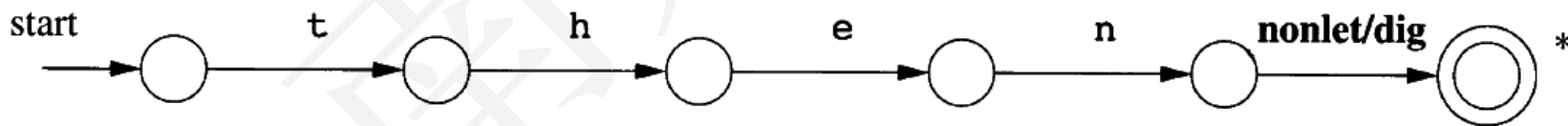


图 3-15 假想的关键字 then 的状态转换图

其它的状态转换图

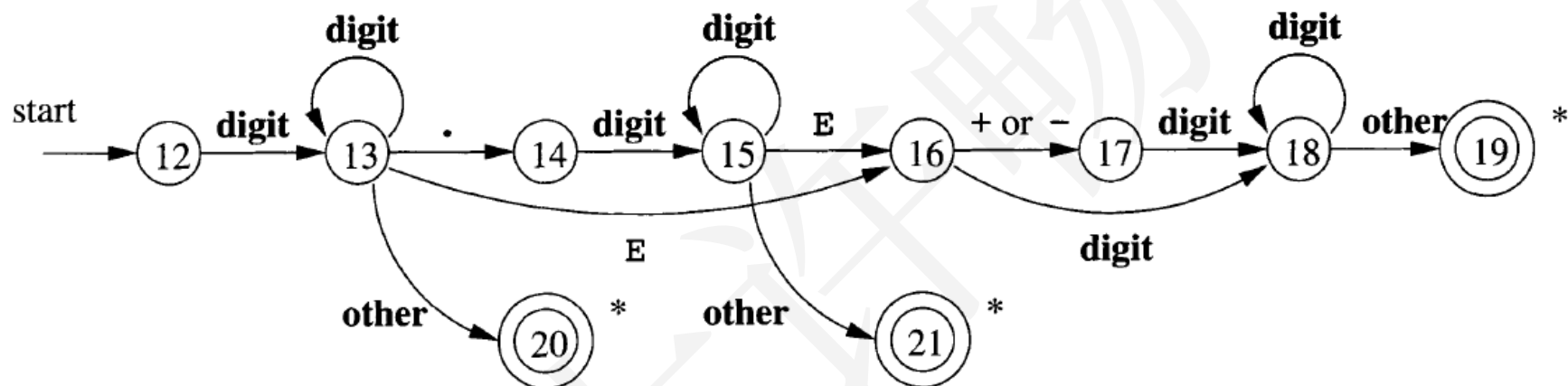


图 3-16 无符号数字的状态转换图

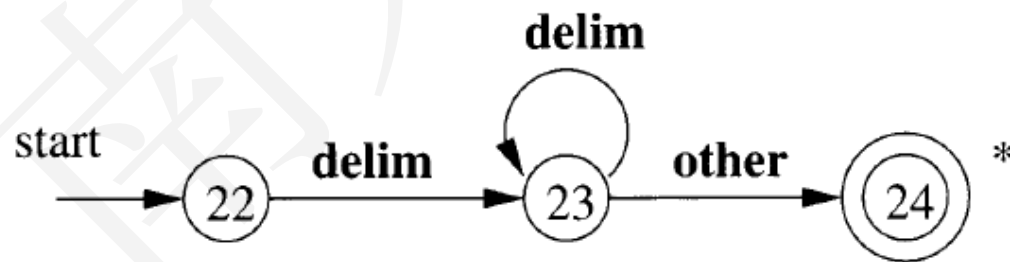


图 3-17 空白符的状态转换图

词法分析器的体系结构

- 从转换图构造词法分析器的方法
 - 变量**state**记录当前状态
 - 一个**switch**语句根据**state**的值转到相应的代码
 - 每个状态对应于一段代码
 - 这段代码根据读入的符号，确定下一个状态
 - 如果找不到相应的边，则调用**fail()**进行错误恢复
 - 进入某个接受状态时，返回相应的词法单元
 - 注意状态有*标记时，需要回退**forward**指针
- 实际是模拟转换图的运行

relop对应的代码概要

```
TOKEN getRelop()
{
    TOKEN retToken = new(RELOP);
    while(1) { /* repeat character processing until a return
                or failure occurs */
        switch(state) {
            case 0: c = nextChar();
                    if ( c == '<' ) state = 1;
                    else if ( c == '=' ) state = 5;
                    else if ( c == '>' ) state = 6;
                    else fail(); /* lexeme is not a relop */
                    break;
            case 1: ...
            ...
            case 8: retract();
                    retToken.attribute = GT;
                    return(retToken);
        }
    }
}
```

处理多个模式的方法

- 词法分析器需要匹配多个模式
- 解决方法
 - 按照**优先级**，顺序地尝试各个状态转换图，如果引发fail()，回退并尝试下一个状态图
 - 更好的方法：**并行**地运行各个状态转换图；通过greedy策略，识别**最长**的与某个模式匹配的输入前缀
 - 实际使用的方法：预先把各个状态转换图合成一个状态转换图，然后运行这个状态转换图 (后面介绍)

内容

- 词法分析器的作用
- 词法单元的规约 (正则表达式)
- 词法单元的识别 (状态转换图)
- 词法分析器生成工具及设计
- 有穷自动机

词法分析工具Lex

- Lex/Flex是一个有用的词法分析器生成工具
- 通常和Yacc一起使用，生成编译器的前端

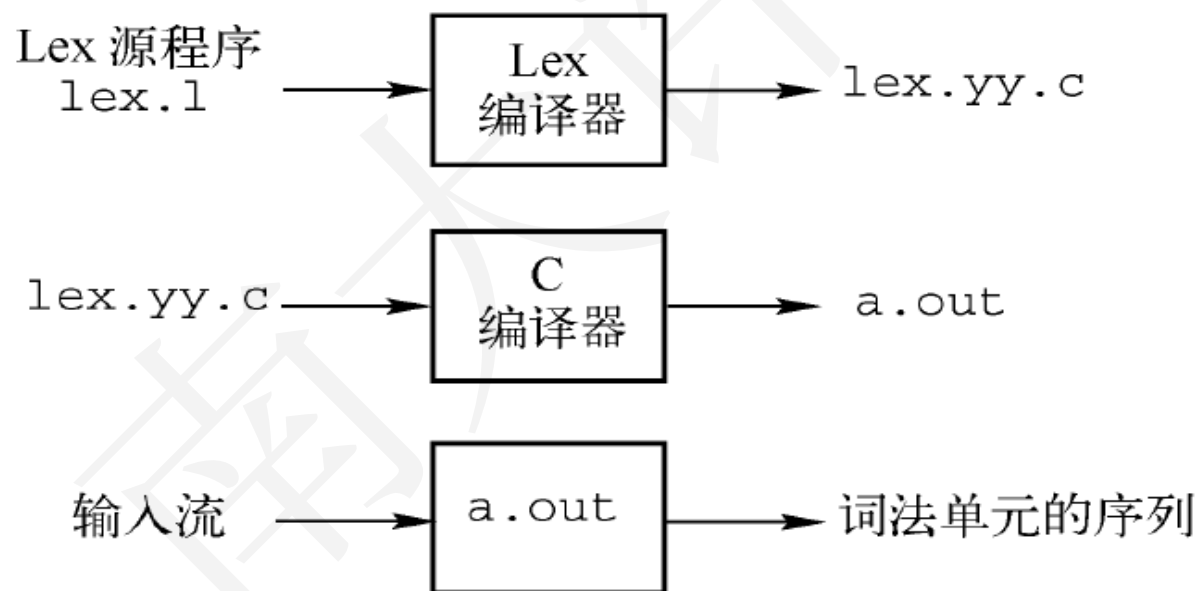


图 3-22 用 Lex 创建一个词法分析器

Lex源程序的结构

- 声明部分
 - 常量：表示常数的标识符
 - 正则定义
- 转换规则
 - 模式 { 动作 }
 - 模式是正则表达式
 - 动作表示识别到相应模式时应采取的处理方式
 - 处理方式通常用是C语言代码表示
- 辅助函数
 - 各个动作中使用的函数

声明部分

%%

转换规则

%%

辅助函数

Lex程序的形式

例子 (1)

```
%{  
    /* definitions of manifest constants  
    LT, LE, EQ, NE, GT, GE,  
    IF, THEN, ELSE, ID, NUMBER, RELOP */  
}%  
  
/* regular definitions */  
delim      [ \t\n]  
ws         {delim}+  
letter     [A-Za-z]  
digit      [0-9]  
id         {letter}({letter}|{digit})*  
number     {digit}+(\.{digit}+)?(E[+-]?{digit}+)?  
  
%%
```

%{和}%之间的内容一般被直接拷贝到lex.yy.c中；
这里的内容就是一段注释；
LT和LE等值在Yacc源程序中定义

正则定义

分隔声明部分和
转换规则部分

例子 (2)

%%

```
{ws}      { /* no action and no return */ }
if         {return(IF);}
then       {return(THEN);}
else       {return(ELSE);}
{id}       {yylval = (int) installID(); return(ID);}
{number}   {yylval = (int) installNum(); return(NUMBER);}
"<"        {yylval = LT; return(RELOP);}
"<="       {yylval = LE; return(RELOP);}
"="        {yylval = EQ; return(RELOP);}
"<>"       {yylval = NE; return(RELOP);}
">"        {yylval = GT; return(RELOP);}
">="       {yylval = GE; return(RELOP);}
```

%%

没有返回，表示
继续识别其它的
词法单元

把识别到的标识符
加入标识符表

识别到数字常量，
加入常量表

例子 (3)

- Lex处理源程序时，辅助函数被拷贝到lex.yy.c中
- 辅助函数可在规则中直接调用

%%

```
int installID() { /* function to install the lexeme, whose  
                  first character is pointed to by yytext,  
                  and whose length is yyleng, into the  
                  symbol table and return a pointer  
                  thereto */
```

```
}
```

```
int installNum() { /* similar to installID, but puts numer-  
                    ical constants into a separate table */
```

```
}
```

词法分析器的工作方式

- Lex生成的词法分析器作为一个函数被调用
- 在每次调用过程中，不断读入余下的输入符号
- 发现最长的、与某个模式匹配的输入前缀时
 - 调用相应的动作，该动作进行相关处理
 - 之后词法分析器继续寻找其它词素

Lex中的冲突解决方法

- **冲突**：多个输入前缀与某个模式相匹配，或者一个前缀与多个模式相匹配
- **Lex解决冲突的方法**
 - 多个前缀可能匹配时，选择**最长的**前缀
 - 比如，词法分析器把<=当作一个词法单元识别
 - 最长的前缀与多个模式匹配时，选择列在**前面的**模式
 - 如果保留字的规则在标识符的规则之前，词法分析器将识别出保留字

内容

- 词法分析器的作用
- 词法单元的规约 (正则表达式)
- 词法单元的识别 (状态转换图)
- 词法分析器生成工具及设计
- 有穷自动机

有穷自动机

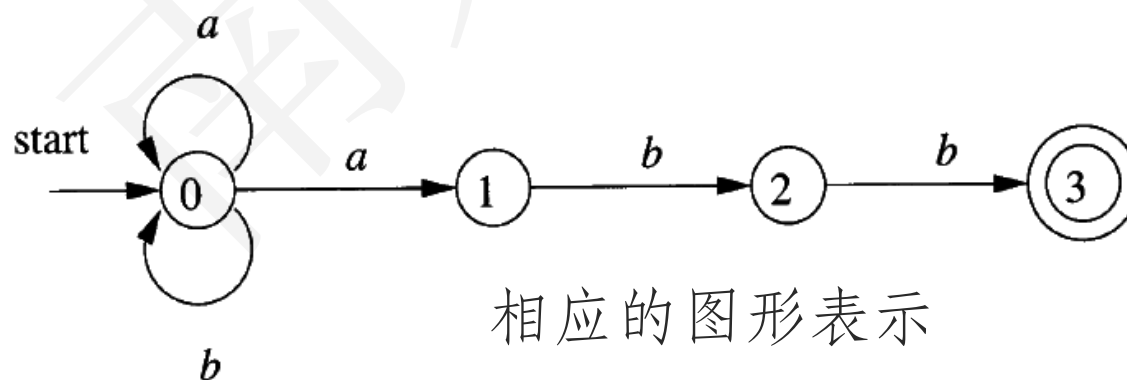
- 本质上和状态转换图相同，但有穷自动机 (finite automata) 只回答 Yes/No
 - 分为两类
 - 不确定的有穷自动机 (Nondeterministic Finite Automata / NFA): 边上的标号没有限制，一个符号可出现在离开同一个状态的多条边上， ϵ 可以做标号
 - 确定的有穷自动机 (Deterministic Finite Automata / DFA): 对于每个状态以及每个符号，有且只有一条边 (或最多只有一条边)
- 两种自动机都识别正则语言 (regular language)
 - 对于每个可以用正则表达式描述的语言，均可用某个NFA或DFA来识别；反之亦然

不确定的有穷自动机

- NFA的定义
 - 一个有穷的状态集合 S
 - 一个输入符号集合 Σ ，即输入字母表 (input alphabet)
 - 转换函数 (transition function) 对于每个状态和 $\Sigma \cup \{\varepsilon\}$ 中的符号，给出相应的后继状态 (next state) 集合
 - S 中的某个状态 s_0 被指定为开始状态/初始状态 (有些定义中可以有多多个开始状态)
 - S 的一个子集 F 被指定为接受状态集合

NFA的例子

- 状态集合 $S = \{ 0, 1, 2, 3 \}$
- 开始状态 s_0 : 0
- 接受状态集合 F : $\{ 3 \}$
- 转换函数
 - $(0, a) \rightarrow \{ 0, 1 \}$ $(0, b) \rightarrow \{ 0 \}$
 - $(1, b) \rightarrow \{ 2 \}$ $(2, b) \rightarrow \{ 3 \}$



转换表 (transition table) 表示法

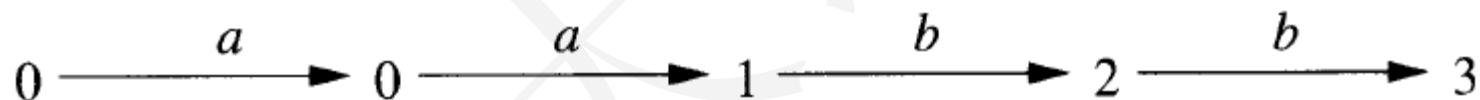
- 用二维表表示NFA的转换函数
 - 每行对应于一个状态
 - 每列对应于一个输入符号或者 ϵ
 - 每个条目表示对应的后继状态集合

状态	a	b	ϵ
0	$\{0, 1\}$	$\{0\}$	$\emptyset$
1	$\emptyset$	$\{2\}$	$\emptyset$
2	$\emptyset$	$\{3\}$	$\emptyset$
3	$\emptyset$	$\emptyset$	$\emptyset$

转换表表示法

输入字符串的接受

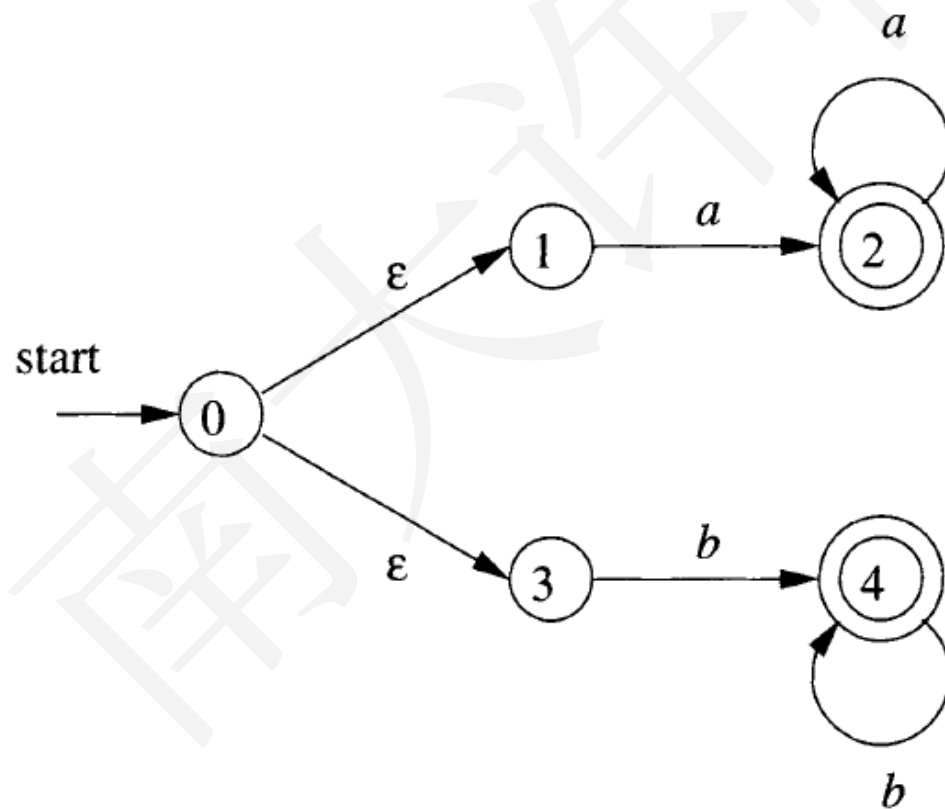
- 一个NFA接受 (accept) 输入字符串 x
 - 当且仅当对应的转换图中存在一条从开始状态到某个接受状态的路径，且该路径各条边上的标号按顺序组成 x (不含 ϵ 标号)
- 前面的NFA接受 aabb，因为



- NFA接受的语言：从开始状态到达接受状态的所有路径的标号串的集合
 - 即该NFA接受的所有符号串的集合

NFA和相应语言的例子

- 相应的语言: $L(aa^* \mid bb^*)$



确定的有穷自动机

- 一个NFA被称为DFA，如果
 - 没有标号为 ϵ 的转换，并且
 - 对于每个状态 s 和每个输入符号 a ，有且仅有一条标号为 a 的离开 s 的边
- 可以高效判断一个串能否被一个DFA接受
- 每个NFA都有一个等价的DFA (后面介绍)
 - 即它们接受相同的语言

DFA的模拟运行

- 假设输入符号是字符
- *nextChar*读入下一个字符
- *move*给出了离开状态*s*且标号为*c*的边的目标状态

```
s = s0;  
c = nextChar();  
while ( c != eof ) {  
    s = move(s, c);  
    c = nextChar();  
}  
if ( s 在 F 中 ) return "yes";  
else return "no";
```


DFA的例子

- 假设输入为 **ababb**，那么进入的状态序列为
 - 0, 1, 2, 1, 2, 3，返回yes

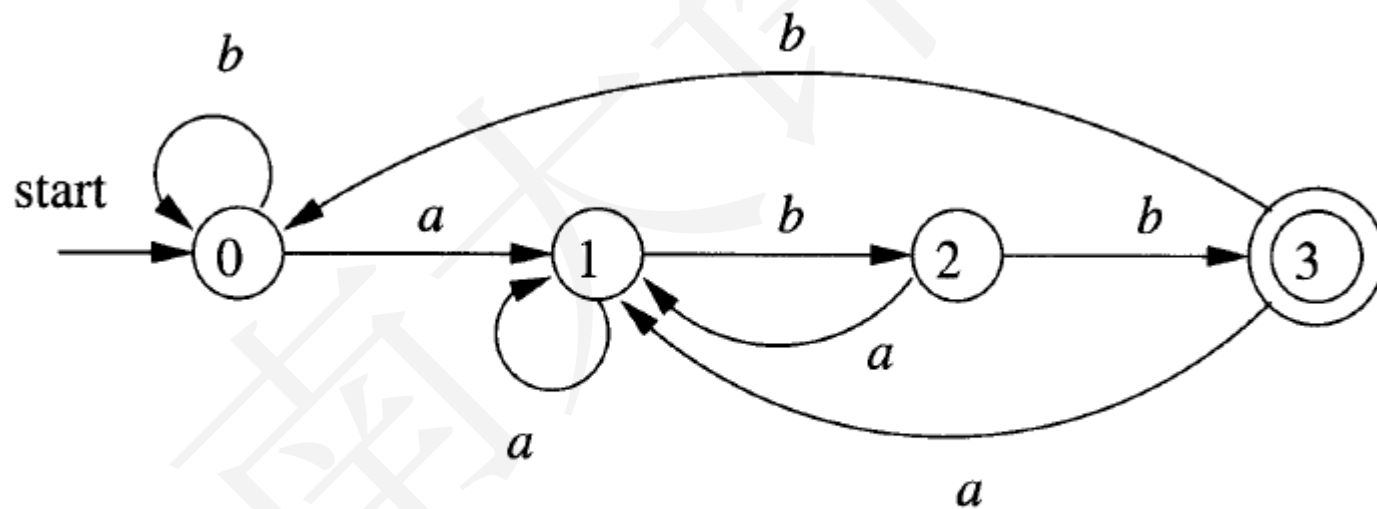


图 3-28 接受 $(a|b)^*abb$ 的 DFA

从正则表达式到自动机的转换

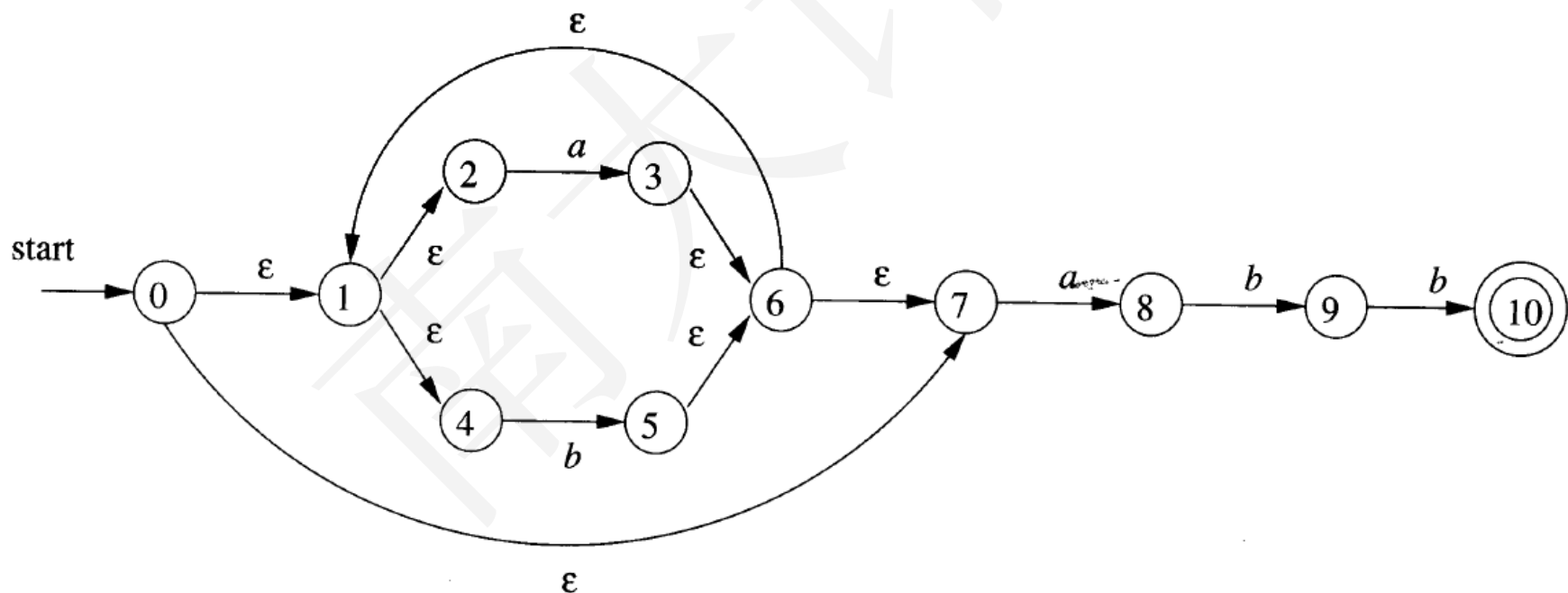
- 正则表达式可简洁、**精确**地描述词法单元的模式
- 模拟DFA的执行可**高效**地进行模式匹配
- 将正则表达式转换为DFA的步骤
 - 正则表达式到NFA (后讲)
 - NFA到DFA

NFA 到 DFA (1)

- 子集构造法 (subset construction) 的基本思想
 - 构造得到的DFA的每个状态和NFA的状态子集对应
 - DFA读入 $a_1, a_2, \dots, a_n$ 后到达的状态对应于从NFA开始状态出发沿着 $a_1, a_2, \dots, a_n$ 可能到达的状态集合
 - 在算法中“并行地模拟”NFA在遇到一个给定输入串时可能执行的所有动作

例子 (1)

- 下面的NFA能够接受串babb
 - 考虑从开始状态出发，沿着标号分别为b, ba, bab, babb能到达的所有可能状态的集合



NFA 到 DFA (2)

- 理论上，最坏情况下DFA的状态个数会是NFA状态个数的指数多个
 - 但对于大部分应用，NFA和相应的DFA的状态数量大致相同

NFA 到 DFA (3)

- 算法中使用到的基本操作
 - ϵ -closure(s): 从NFA状态 s 开始, 只通过 ϵ 转换能到达的NFA状态集合
 - ϵ -closure(T): 从 T 中某个状态 s 开始, 只通过 ϵ 转换能到达的NFA状态集合
 - move(T, a): 从 T 中某个状态 s 出发, 通过一个标号为 a 的转换能到达的NFA状态集合

NFA 到 DFA (4)

- 计算 ϵ -closure(T) 的算法
 - 实际上是一个图搜索过程 (只考虑 ϵ 标号边)

```
将  $T$  的所有状态压入  $stack$  中;  
将  $\epsilon$ -closure( $T$ ) 初始化为  $T$ ;  
while (  $stack$  非空 ) {  
    将栈顶元素  $t$  弹出栈中;  
    for ( 每个满足如下条件的  $u$ : 从  $t$  出发有一个标号为  $\epsilon$  的转换到达状态  $u$  )  
        if (  $u$  不在  $\epsilon$ -closure( $T$ ) 中 ) {  
            将  $u$  加入到  $\epsilon$ -closure( $T$ ) 中;  
            将  $u$  压入栈中;  
        }  
}
```

NFA 到 DFA (5)

- 整个算法实际是一个搜索过程
 - $Dstates$ 中的一个状态未加标记表示还没有搜索过它的各个后继

一开始, $\epsilon\text{-closure}(s_0)$ 是 $Dstates$ 中的唯一状态, 且它未加标记;

```
while ( 在  $Dstates$  中有一个未标记状态  $T$  ) {  
    给  $T$  加上标记;  
    for ( 每个输入符号  $a$  ) {  
         $U = \epsilon\text{-closure}(\text{move}(T, a))$ ;  
        if (  $U$  不在  $Dstates$  中 )  
            将  $U$  加入到  $Dstates$  中, 且不加标记;  
         $Dtran[T, a] = U$ ;  
    }  
}
```


例子 (2)

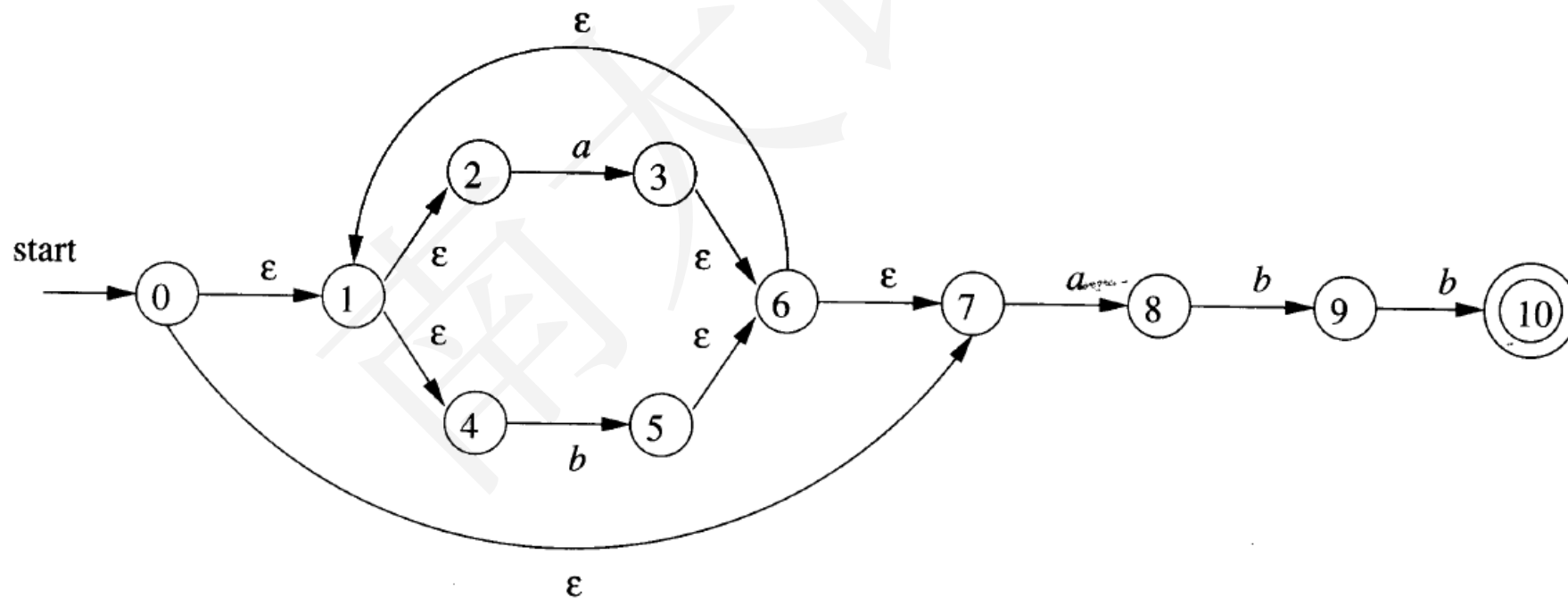
A: ϵ -closure(0) = { 0, 1, 2, 4, 7 }

B: $Dtran[A, a] = \epsilon$ -closure({ 3, 8 }) = { 1, 2, 3, 4, 6, 7, 8 }

C: $Dtran[A, b] = \epsilon$ -closure({ 5 }) = { 1, 2, 4, 5, 6, 7 }

D: $Dtran[B, b] = \epsilon$ -closure({ 5, 9 }) = { 1, 2, 4, 5, 6, 7, 9 }

...



例子 (3)

- 开始状态：A；接受状态：E

NFA 状态	DFA 状态	<i>a</i>	<i>b</i>
<u>0</u> 1, 2, 4, 7}	<i>A</i>	<i>B</i>	<i>C</i>
{1, 2, 3, 4, 6, 7, 8}	<i>B</i>	<i>B</i>	<i>D</i>
{1, 2, 4, 5, 6, 7}	<i>C</i>	<i>B</i>	<i>C</i>
{1, 2, 4, 5, 6, 7, 9}	<i>D</i>	<i>B</i>	<i>E</i>
{1, 2, 4, 5, 6, 7, <u>10</u> }	<i>E</i>	<i>B</i>	<i>C</i>

图 3-35 DFA *D* 的转换表 *Dtran*

正则表达式到NFA

- 基本思想
 - 根据正则表达式的递归定义，按照正则表达式的结构递归地构造出相应的NFA
 - 算法分成两个部分
 - 基本规则处理 ϵ 和单符号的情况
 - 对于每个正则表达式的运算，建立组合相应NFA的方法

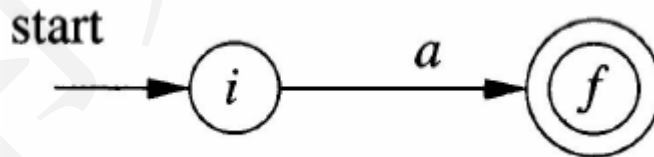
转换算法 (1)

- 基本规则部分

- 表达式 ϵ



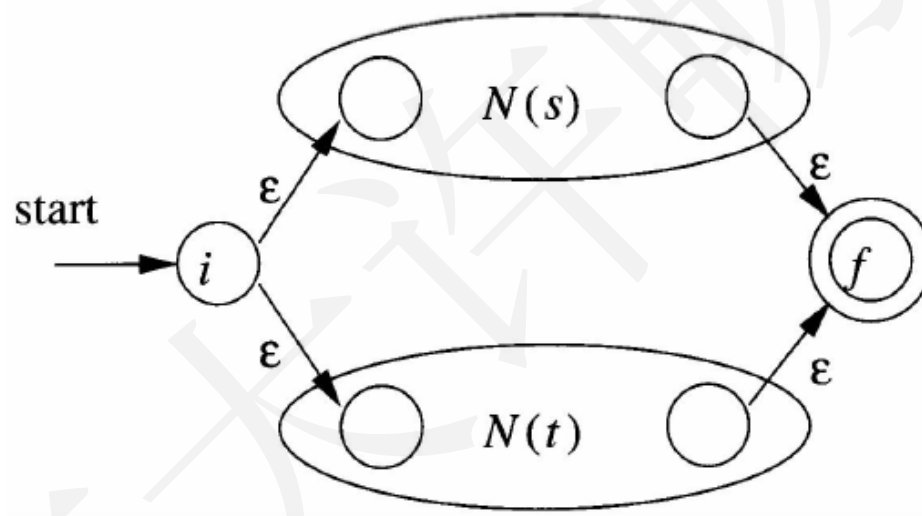
- 表达式 a



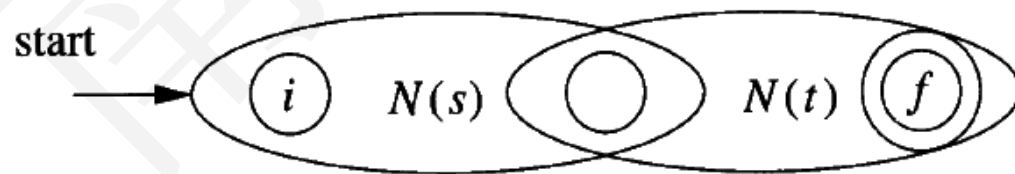
转换算法 (2)

- 归纳部分

- $s \mid t$

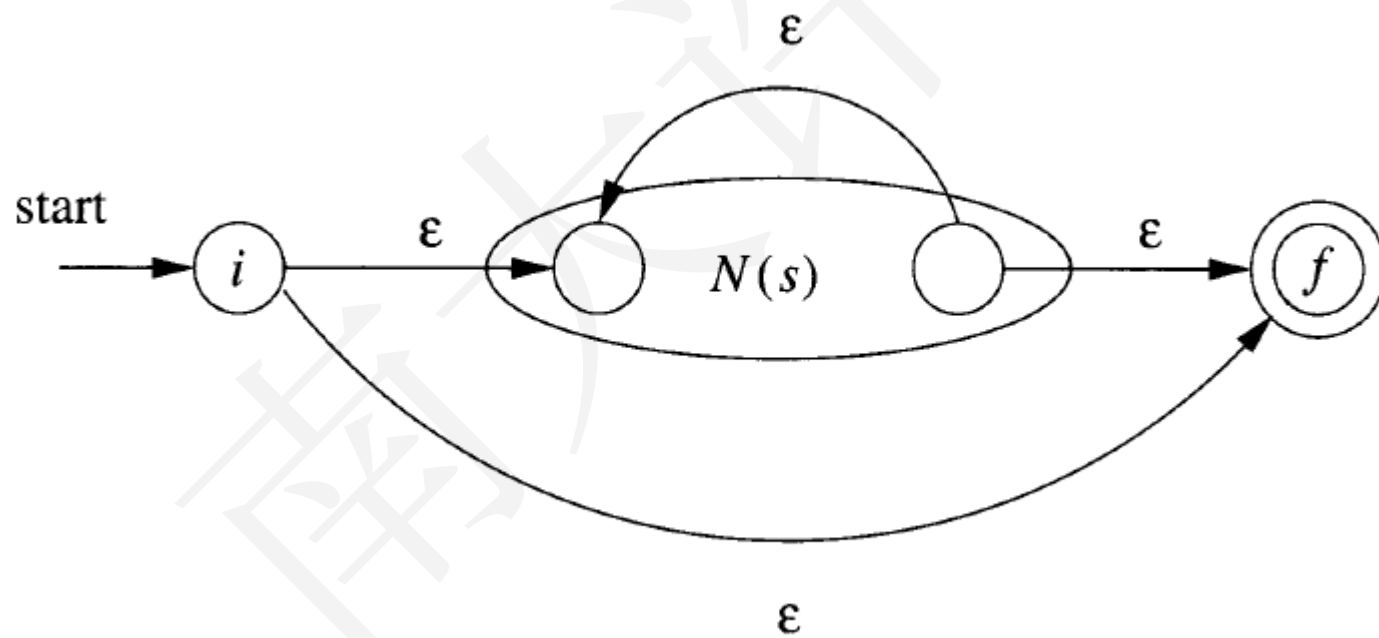


- st



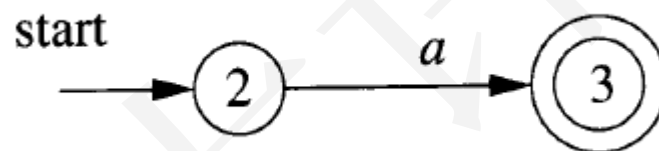
转换算法 (3)

- 归纳部分
 - s^*

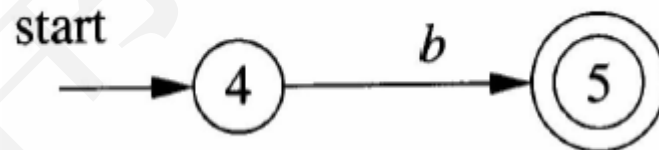


例子 (1)

- 正则表达式: $(a|b)^*abb$
- 第一个a对应的NFA

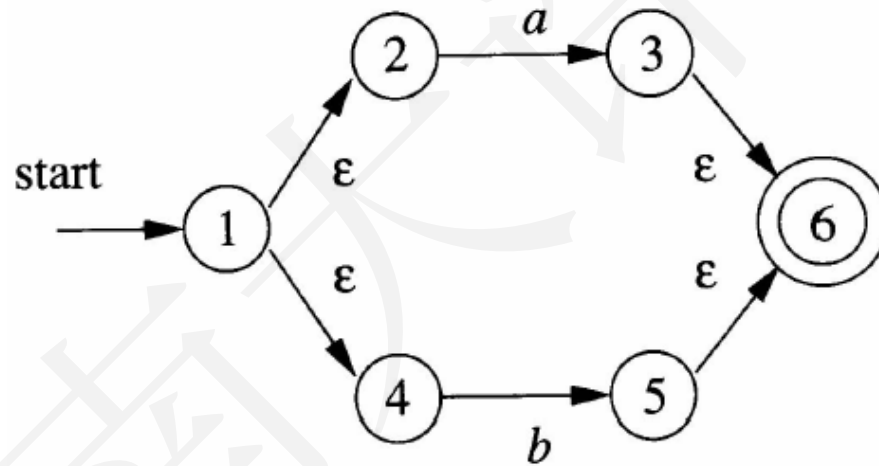


- 第一个b对应的NFA



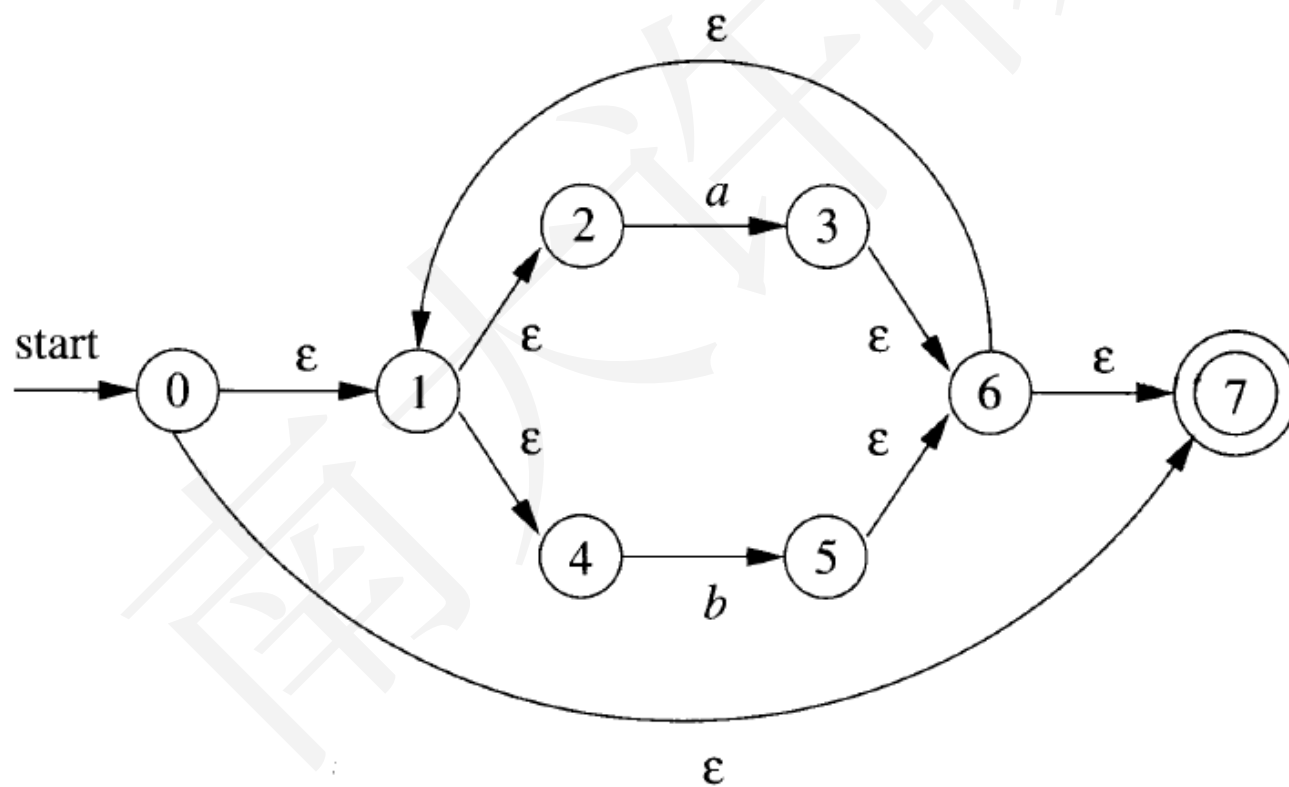
例子 (2)

- $(a|b)$ 的 NFA



例子 (3)

- $(a|b)^*$ 的 NFA

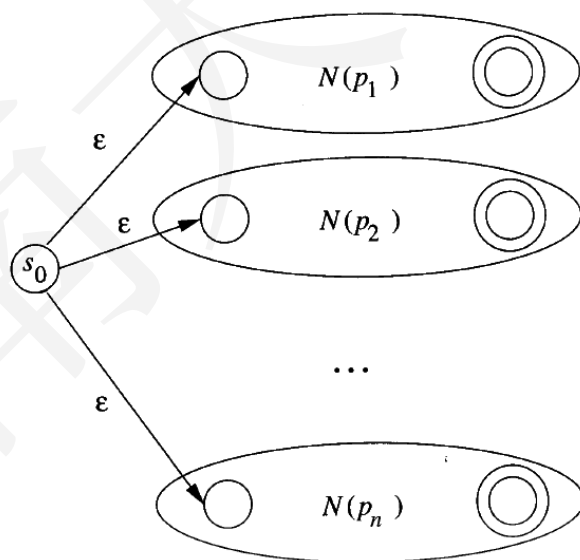


词法分析器生成工具的功能

- 生成的词法分析器中包含一个模拟有穷自动机的模块
- 其余部分由生成工具根据词法规则的描述自动生成，包括
 - 自动机的转换表
 - 与动作相关的代码，适当的时候由模拟器调用
- 构造自动机时
 - 首先构造出各个模式对应的NFA
 - 然后将这些NFA合并成为一个NFA
 - (根据需要) 进行确定化 (DFA)

NFA合并的方法

- 合并方法
 - 引入新的开始状态，并引入从该开始状态到各个原开始状态的 ϵ 转换
 - 得到的NFA所接受的语言是原来各个NFA语言的并集
 - 不同的接受状态代表不同的模式



确定化NFA后的处理

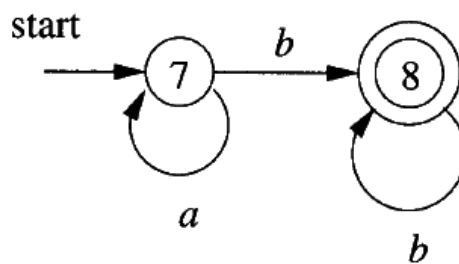
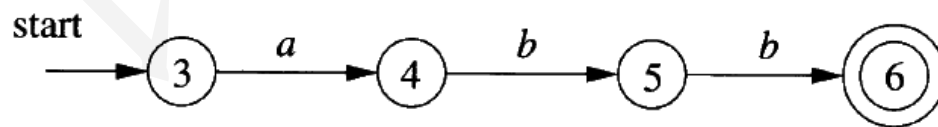
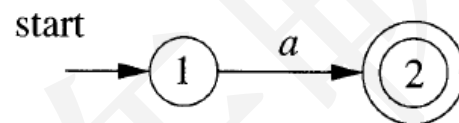
- 对得到的NFA进行确定化，得到DFA
- 一个DFA的接受状态对应于NFA的状态子集，其中至少包括一个NFA的接受状态
 - 如果其中包括多个对应于不同模式的NFA接受状态，则表示当前的输入前缀对应于多个模式，存在冲突
 - 找出第一个这样的模式，将该模式作为此DFA接受状态的输出

例子 (1)

- 假设有三个模式

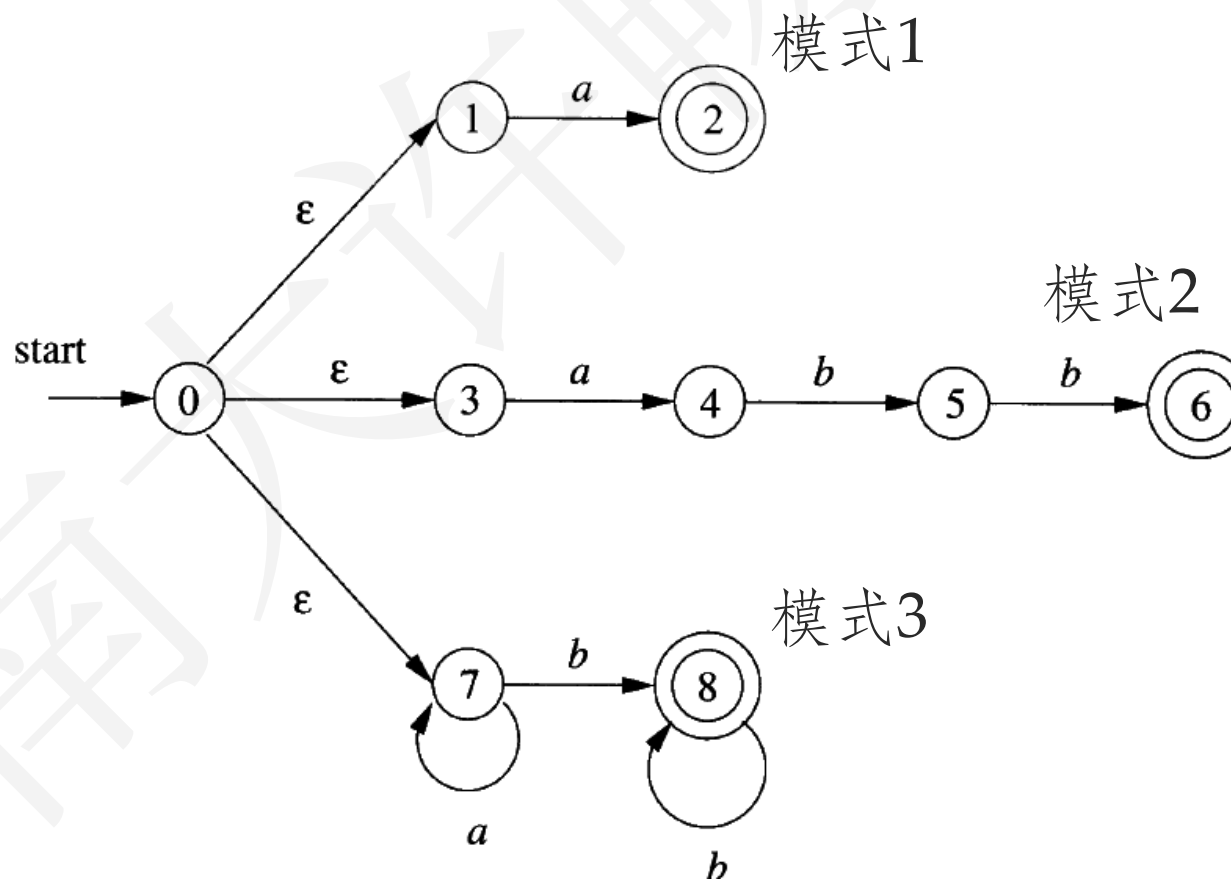
- $a \{ A1 \}$
- $abb \{ A2 \}$
- $a^*b^+ \{ A3 \}$

- 构造各模式的NFA



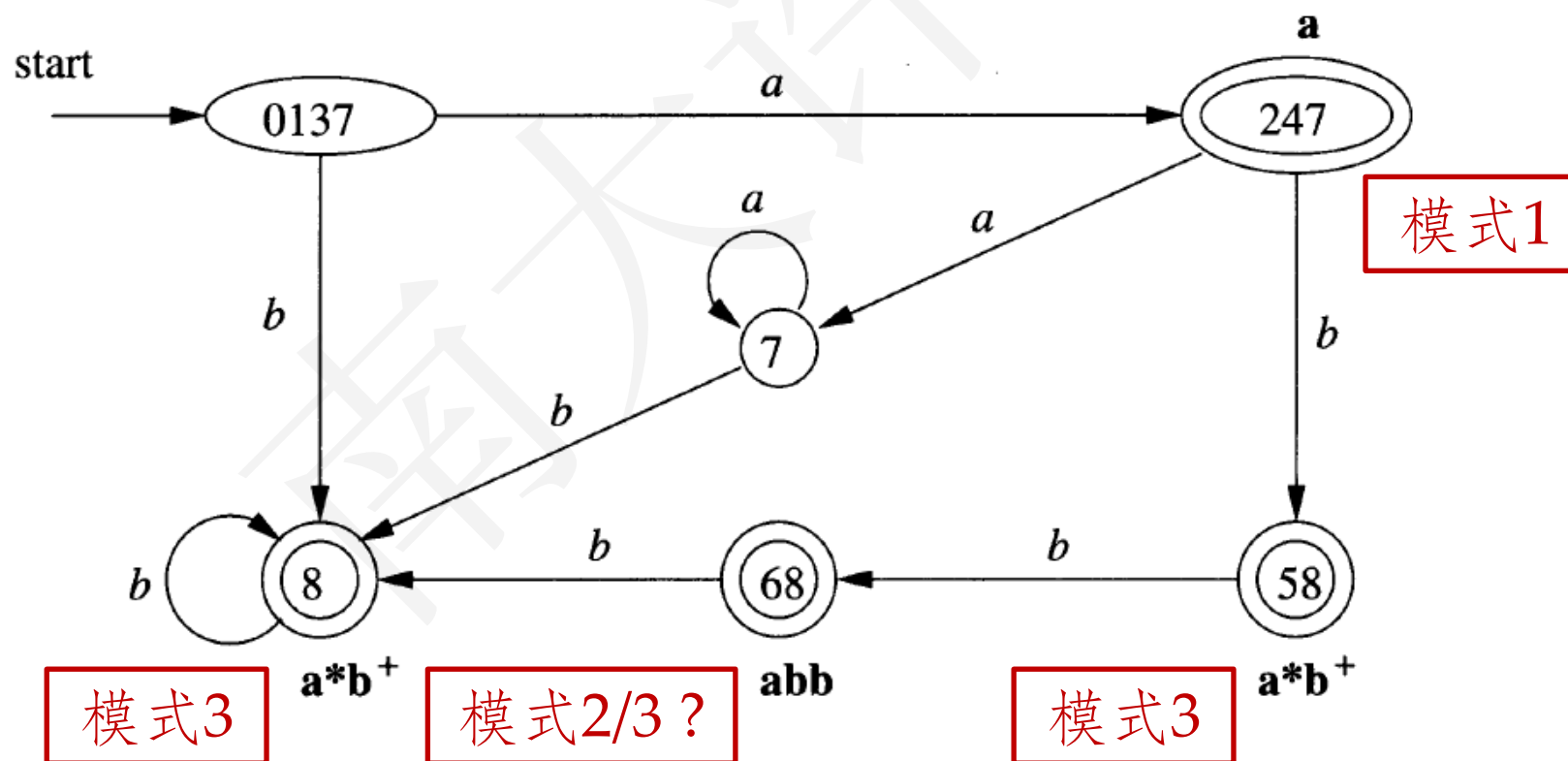
例子 (2)

- 合并NFA
 - 2: 模式1
 - 6: 模式2
 - 8: 模式3



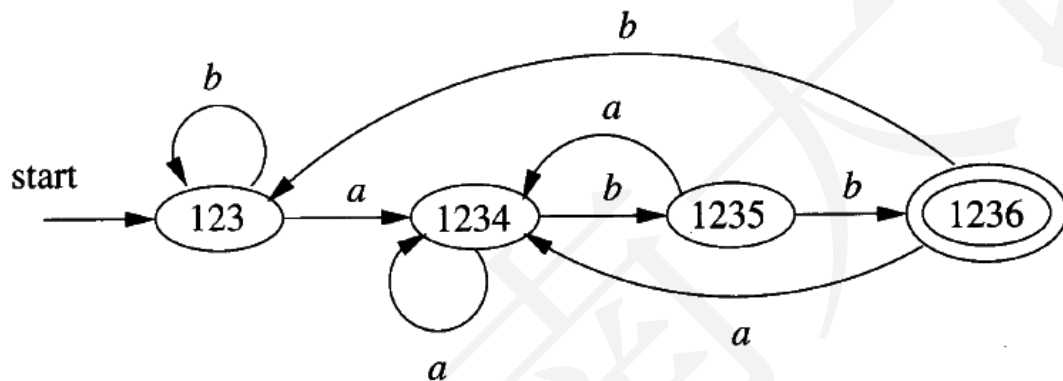
例子 (3)

- 确定化得到如下DFA
 - DFA状态68对应的NFA状态子集为{ 6, 8 }, 其对应的模式是abb (即模式2), 而不是 a^*b^+ (即模式3)

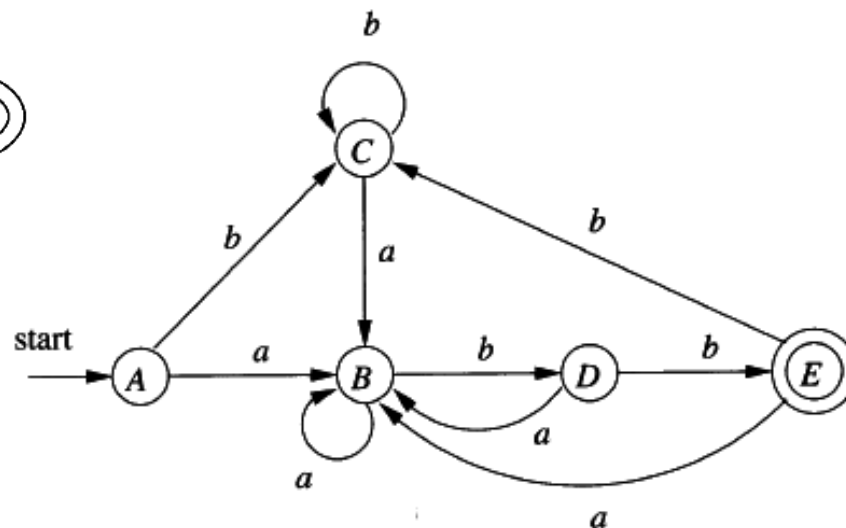


DFA状态数量的最小化

- 一个正则语言可对应于多个识别此语言的DFA
- 通过**DFA的最小化**可得到**状态数量最少**的DFA (不计同构, 这样的DFA是**唯一**的)

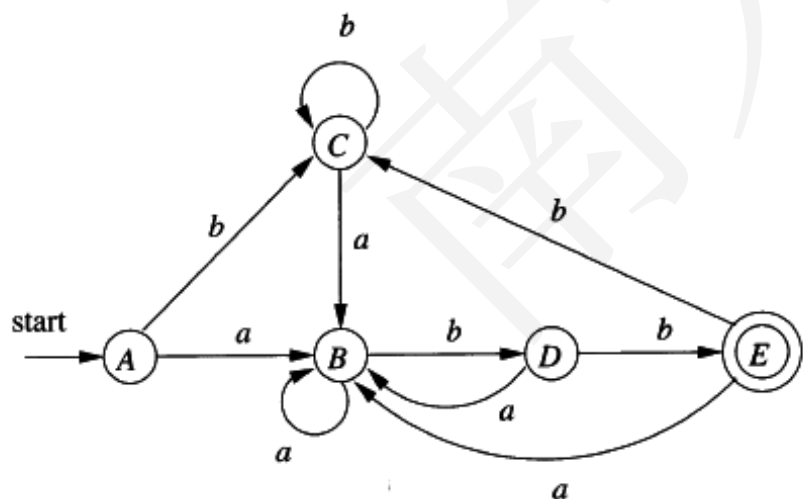


两个等价的DFA: 都识别
 $(a|b)^*abb$



状态的区分

- 状态的可区分 (distinguishable)
 - 如果存在串 x , 使得从状态 s_1 和 s_2 , 一个到达接受状态而另一个到达非接受状态, 那么 x 就区分了 s_1 和 s_2
 - 如果存在某个串区分了 s 和 t , 我们说 s 和 t 是可区分的, 否则它们是不可区分的
- 不可区分的两个状态就是等价的, 可以合并



空串区分了E和其它状态;
bb区分了A和B

DFA最小化算法

- 把所有可区分的状态分开 (迭代过程)
 - 基本步骤: ϵ 区分了接受状态和非接受状态
 - 归纳步骤: 如果 s 和 t 是可区分的, 且 s' 到 s 、 t' 到 t 有标号为 a 的边, 那么 s' 和 t' 也是可区分的
- 最终没有区分开的状态就是等价的
 - 所有的死状态都是等价的
- 从划分得到的等价类中选取代表, 并重建DFA

最小化算法 (划分部分)

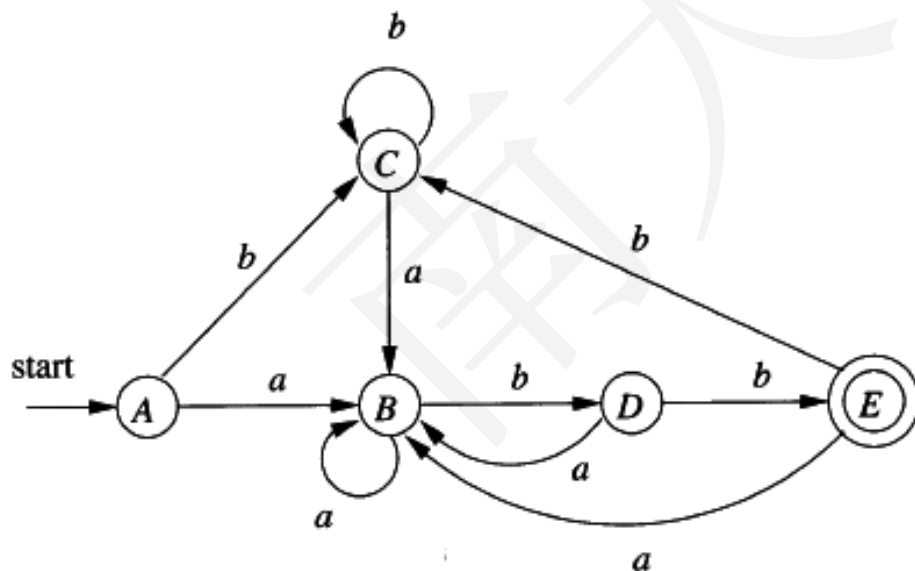
- 设置初始划分: $\Pi = \{ S - F, F \}$
- 迭代, 不断划分
 - for (Π 中的每个元素 G) {
 - 细分 G , 使得 G 中的 s 、 t 仍然在同一组中 iff
 - 对任意输入 a , s 、 t 都到达 Π 中的同一组;
 - $\Pi_{\text{new}} =$ 将 Π 中的 G 替换为细分得到的小组}
- 如果 $\Pi_{\text{new}} == \Pi$, 令 $\Pi_{\text{final}} = \Pi$, 算法完成; 否则 $\Pi = \Pi_{\text{new}}$, 转步骤2

最小化算法 (构造部分)

- 在 Π_{final} 的每个组中选择一个状态作代表，作为最小化DFA中的状态
 - 开始状态就是包含原开始状态的组的代表
 - 接受状态就是包含了原接受状态的组的代表 (这个组一定只包含接受状态)
 - 转换关系构造如下
 - 如果 s 是 G 的代表，而原DFA中 s 在 a 上的转换到达 t ，且 t 所在组的代表为 r ，那么最小化DFA中有从 s 到 r 的在 a 上的转换

DFA最小化的例子

- 初始划分: $\{\{A, B, C, D\}, \{E\}\}$
- 处理 $\{A, B, C, D\}$: b 把它细分为 $\{A, B, C\}$ 和 $\{D\}$
- 处理 $\{A, B, C\}$: b 把它细分为 $\{A, C\}$ 和 $\{B\}$
- 选取 A, B, D 和 E 为代表, 构造得到最小DFA



状态	a	b
A	B	A
B	B	D
D	B	E
E	B	A

图 3-65 状态最少
DFA 的转换表

词法分析器状态的最小化

- 基本思想和DFA最小化算法相同
- 差别
 - 词法分析器中的接受状态对应于不同的模式
 - 对应不同模式的接受状态一定是不等价的
 - 初始划分为
 - 所有非接受状态集合 + 对应各模式的接受状态集合
- 其余划分和构造的方法均相同
- 接受状态对应的模式就是原来的模式

例子

- 初始划分: $\{0137, 7\}, \{247\}, \{68\}, \{8, 58\}, \{\Phi\}$
 - 增加死状态 Φ

